

WAT4

Web Application Testing

Tracing and Monitoring

P. Studener

Introduction & Motivation

Monitoring, Logging & Tracing

- Understand observability concepts
- Integrate observability into testing workflows
- Use metrics, logs, and traces together
- Identify bottlenecks using distributed tracing
- Interpret monitoring dashboards

Why Monitoring & Tracing?

- Systems are:
 - Distributed
 - Asynchronous
 - Cloud-native
- Failures are complex and non-obvious
- 🙌 Load testing shows symptoms
- 🙌 Monitoring & tracing reveal root causes

From Testing to Observability

- Traditional testing:
 - Pass / Fail
- Modern systems:
 - Continuous behavior analysis
- 🙌 Shift from testing correctness → understanding behavior

Core Concepts

What is Observability?

- Ability to understand system state from outputs

Metrics	Logs	Traces
Aggregated numeric data	Discrete events	Request flows across services

Metrics

- Time-series data
- Examples:
 - CPU usage
 - Request rate
 - Error rate
 - Latency (p95, p99)
- 🙌 Good for alerting & dashboards

Logs

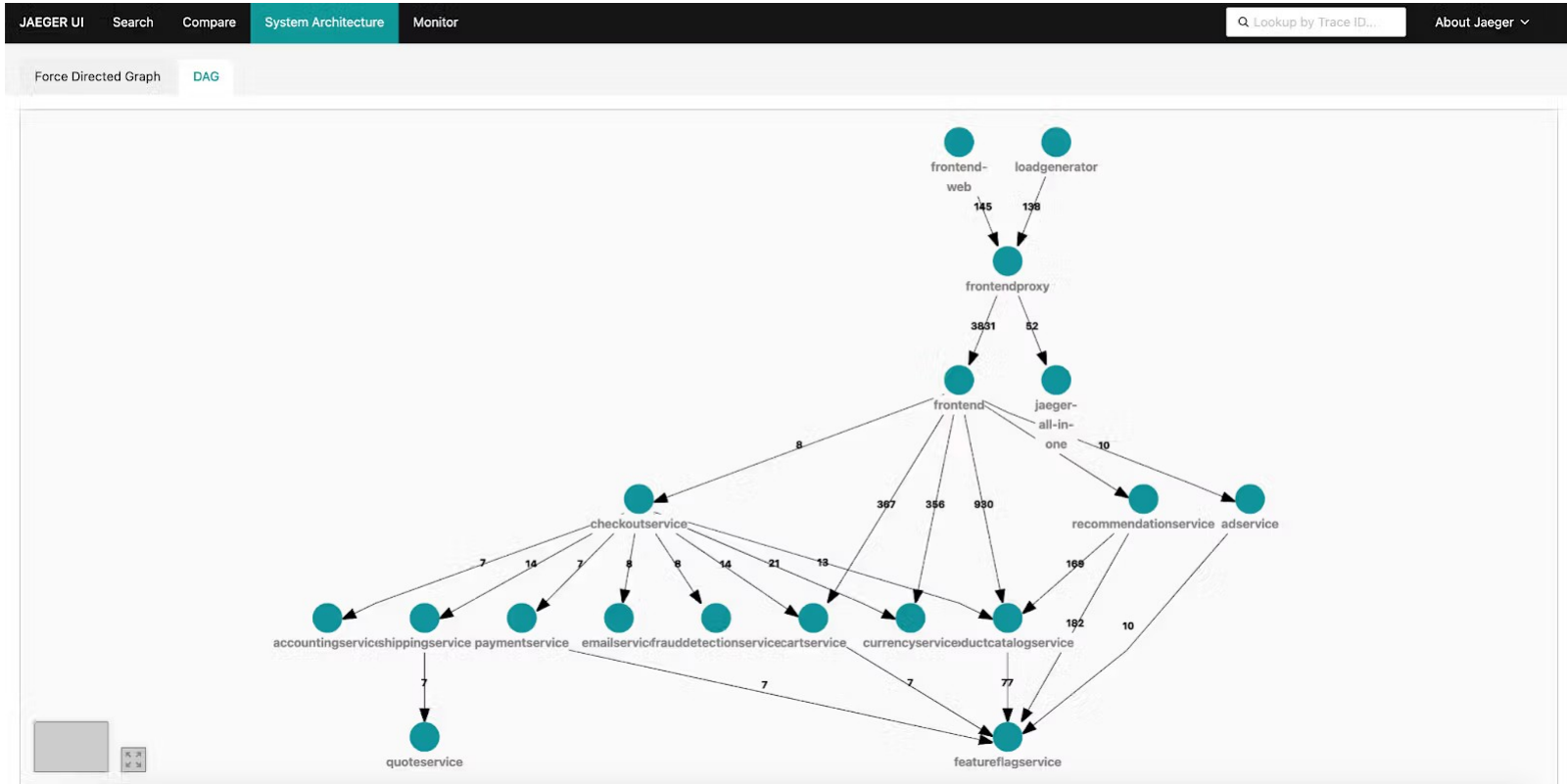
- Timestamped events
- Structured vs. unstructured
- Examples:
 - Errors
 - Warnings
 - Business events
- 🙌 Good for debugging details

Traces

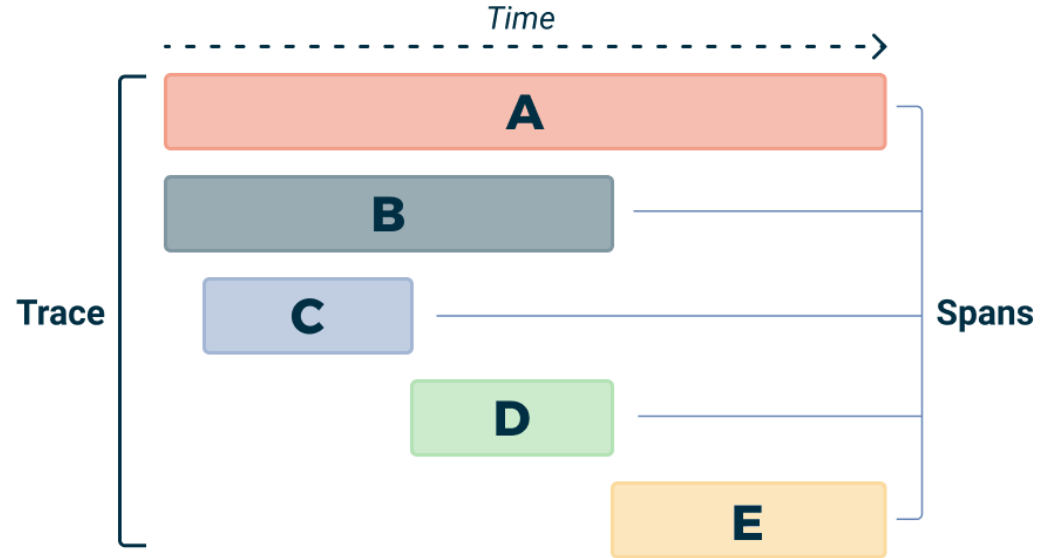
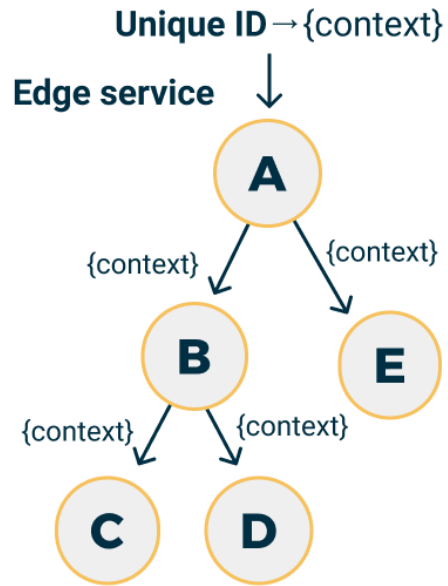
- Track a single request across services
- Show:
 - Latency per service
 - Call hierarchy
 - Dependencies
- 👉 Essential for distributed systems

Distributed Tracing

What is Distributed Tracing?



What is Distributed Tracing?



Trace Structure

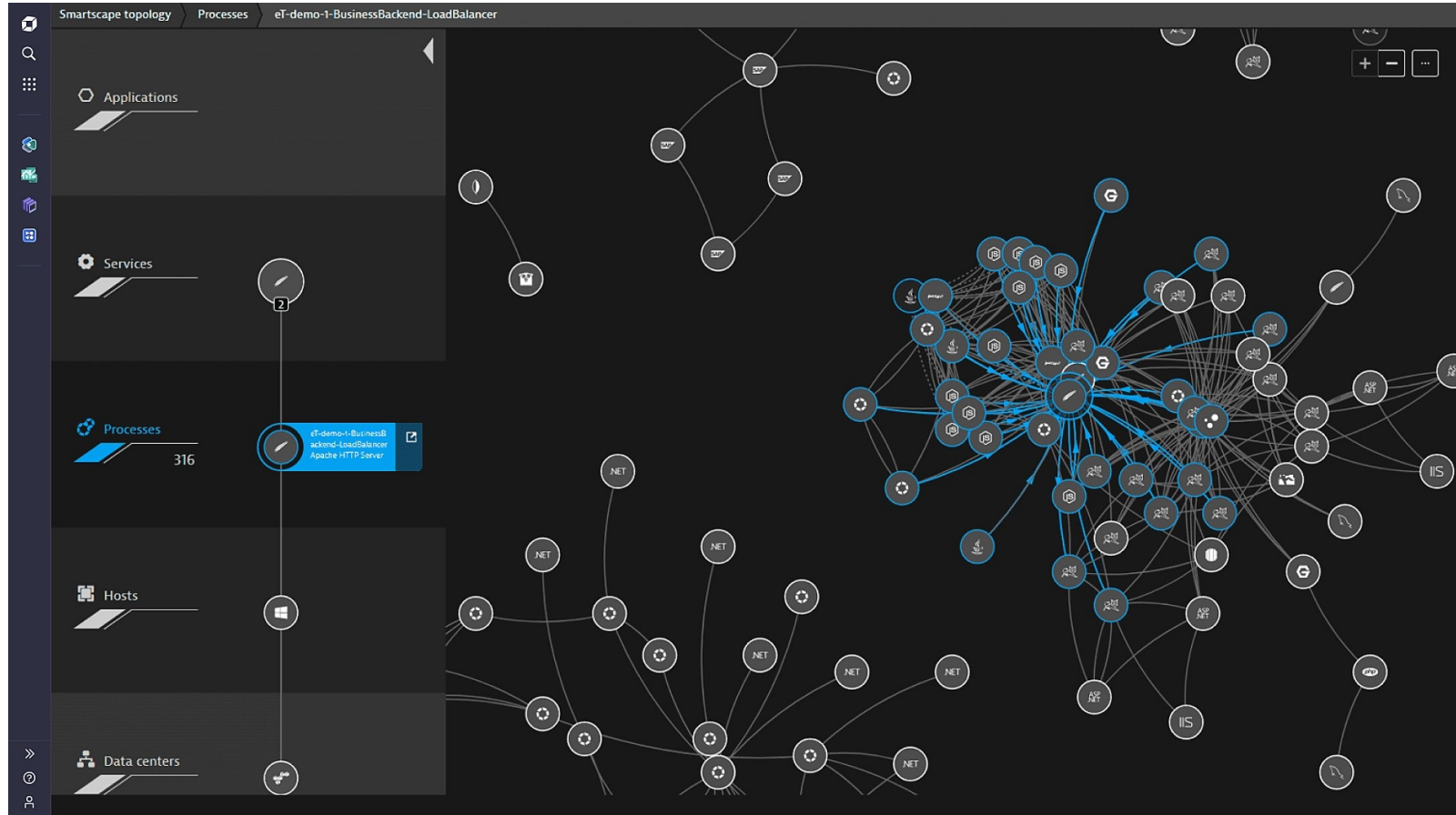
- A trace = end-to-end request
- A span = single operation
- Hierarchy:
 - Root span → child spans
- Trace ID → identifies request
- Span ID → identifies operation
- Parent-child relationships
- Key attributes:
 - Duration
 - Status
 - Metadata (tags)

Example Trace Analysis

- Explain:
 - Slow database query
 - External API delay
 - Bottleneck service
- 🙌 Identify where latency accumulates

Monitoring Systems

Monitoring Architecture



Monitoring Architecture

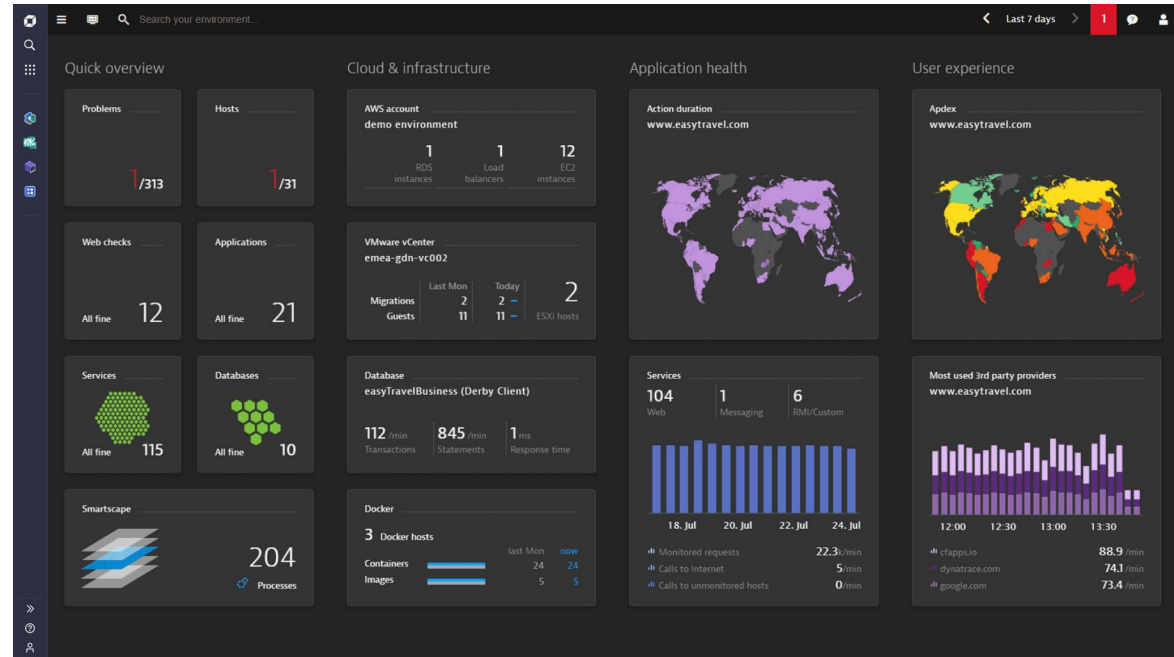
- Application emits data
- Collector gathers data
- Storage persists data
- Visualization dashboards

Metrics Collection

- Pull model (Prometheus)
- Push model (agents)
- Concepts:
 - Scraping
 - Exporters
 - Time-series DB

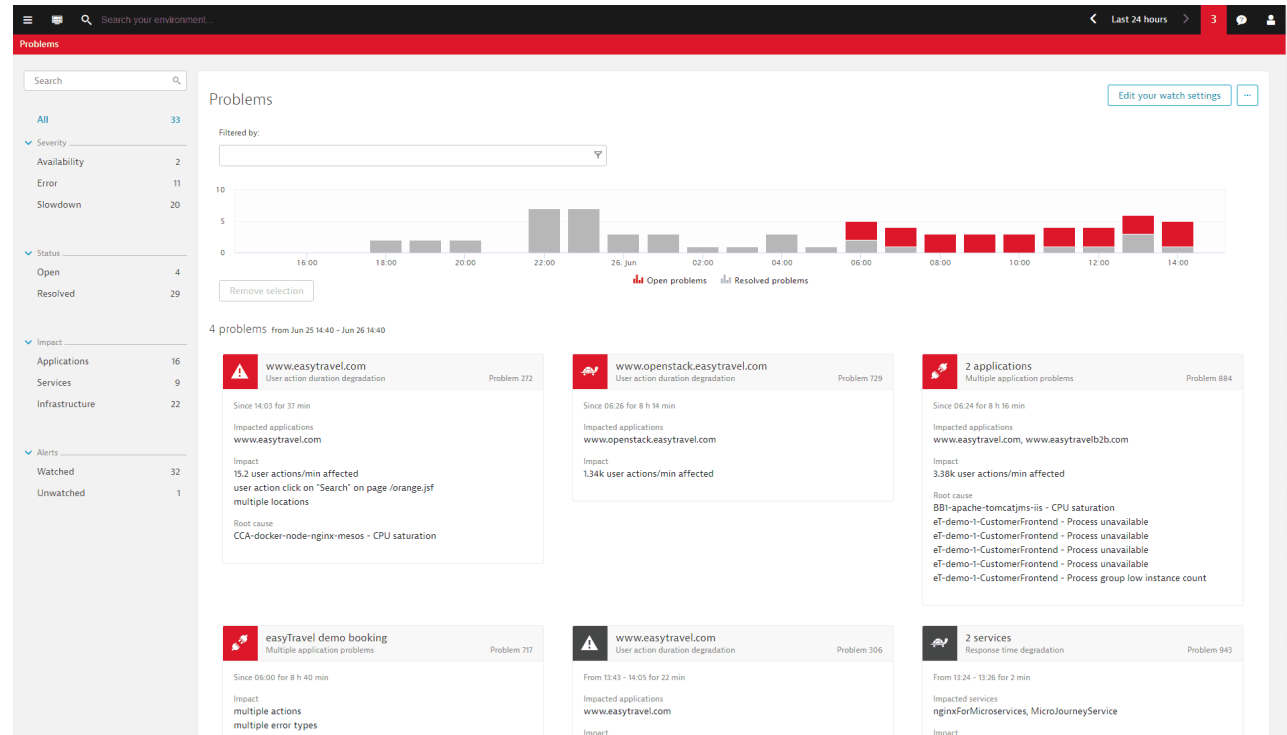
Dashboards

- Visualize:
 - Latency
 - Throughput
 - Errors
- 📌 Good dashboards answer:
 - Is the system healthy?
 - Where is the problem?



Dashboards

- Threshold-based alerts
 - SLO-based alerts
- Examples:
 - p95 latency > 500ms
 - Error rate > 2%



Tooling

Monitoring & Tracing Tools

Monitoring	Tracing
• Prometheus • Grafana • Datadog • New relic • Dynatrace • ...	• Jaeger • Zipkin • OpenTelemetry • ...

OpenTelemetry

- Standard for instrumentation
- Vendor-neutral
- Supports:
 - Metrics
 - Logs
 - Traces
- 🙌 Becoming industry standard

Observability + Load Testing

Why Combine with Load Testing?

- Load test without observability = blind testing
- With monitoring:
 - Identify bottlenecks
 - Understand saturation behavior
 - Validate scaling

Example Workflow

- Run k6 load test
- Observe:
 - CPU / memory
 - DB queries
 - Trace latency
- Identify bottleneck
- Optimize
- Retest

Real-World Use Cases

Detecting Bottlenecks

- High DB latency → indexing issue
- Slow external API → retry/caching
- CPU spikes → inefficient code
- ...

Debugging Incidents

- Alert triggers
- Check dashboard
- Drill down into traces
- Inspect logs
- 🙌 Root cause analysis workflow

Anti-Patterns

Anti-Patterns: Debugging

- No tracing in microservices
- Logging everything (noise)
- Missing correlation IDs
- No alerting
- Dashboards without purpose

Pratical Exercise

Exercise

- Preperation:
 - Add Timeout (Backend)
- Run k6 test
- Observe metrics in Jaeger
- Identify bottleneck